ATLAS VISUEL OBSIDIA / X108 + SRL

V3 Flux - Fusion de 2 documents uniquement

SOURCE 1

audit memoire lot(2).docx

P78, X108, gouvernance kernel, domaines, AgentVote, DomainAggregate, Sigma, Envelope, audit / hash / replay, synthèse kernel.

SOURCE 2

audit memoire ect(2).docx

SRL, mémoire froide, fantôme, session card, triage, indexation, reprise, réactivation, audit SRL readonly.

Objectif : rendre les flux lisibles. Une colonne vertébrale claire par schéma.

Les détails source sont conservés en panneaux latéraux, cartes ou annexes compactes. Pas de gros blocs ASCII.

OUVERTURE

Lire le document comme deux chemins séparés : action gouvernée et continuité mémoire.

Carte globale X108 + SRL

X108	**Domaine** payload / action candidate	**Agents** signalent	**Aggregate** synthétise	**Sigma** surveille	**Kernel** autorise	**Envelope** prouve
SRL	**Session** brute	**Card** fiche	**Triage** classe	**Cold/Ghost** conserve	**Recall** recharge	**Pack** contexte

Message clé

X108 autorise. SRL rappelle. Les deux sont traçables, mais ils n’ont pas la même autorité.

Légende visuelle

Autorité
Kernel X108
seul verrou ALLOW / HOLD / BLOCK

Signal
Agents / votes
observent et signalent

Surveillance
Sigma
stabilité dynamique / proof signals

Preuve
OS3 / audit
hash / replay / attestation

Mémoire
SRL
fiche / triage / rappel

Boundary
readonly / no-write
KX108_ONLY / cannot ACT

Message clé
Une page de flux doit montrer : entrée -> transformation -> gate -> décision -> preuve -> sortie.

PARTIE I - P78

Séparer public, privé, preuve, investisseur, démo, archive et source packs.

P78 - Classification documentaire

DOC_PROOF_TECHNICAL

DOC_AUDIT_TRAIL

DOC_CORE_IMPORT_LEDGER

DOC_FORMAL_PROOF_MAP

DOC_RSSI_EVIDENCE_CANDIDATE

DOC_PUBLIC_SAFE

DOC_PUBLIC_NEEDS_SOFTENING

DOC_INVESTOR_NARRATIVE

DOC_DEMO_ONLY

DOC_PRIVATE_PROPRIETARY

DOC_SOURCE_PACK_LOCAL_ONLY

DOC_ARCHIVE_ONLY

DOC_RUNTIME_INTERNAL

DOC_API_INTERNAL

DOC_MEMORY_INTERNAL

DOC_CONNECTOR_INTERNAL

DOC_DO_NOT_PUBLISH

DOC_UNKNOWN_REVIEW

Message clé

P78 ne doit pas supprimer. P78 ne doit pas déplacer massivement. P78 ne doit pas modifier runtime / Sigma / routes / connectors.

P78 - Principes et décisions possibles

Principes

Support présentation != preuve technique
Public proof != moteur propriétaire complet
Source pack local != canon public
Narratif investisseur != audit RSSI
Démo != runtime souverain
Archive != runtime load

Décisions possibles

KEEP_AS_TECHNICAL_PROOF
KEEP_AS_AUDIT_LEDGER
KEEP_AS_PUBLIC_SAFE
MOVE_LATER_PUBLIC
MOVE_LATER_INVESTOR
MOVE_LATER_DEMO
KEEP_PRIVATE_PROPRIETARY
KEEP_ARCHIVE_ONLY
KEEP_SOURCE_PACK_LOCAL_ONLY
DO_NOT_PUBLISH
REQUIRES_REVIEW

Règles

Docs P56-P77 = audit ledger
specs/_invariant_graph = preuve / architecture, pas pitch
source packs locaux = local-only tant que non canonisés
fichiers avec chemins locaux, routes internes, connectors = pas public safe
preuves techniques liées à tests / commits / manifests / hashes

Message clé

Le split documentaire protège le moteur propriétaire et clarifie ce qui peut nourrir public, investisseur, RSSI ou preuve.

PARTIE II - X108 / Autorité

La structure entre. Les couches signalent. X108 seul autorise.

Vue globale X108

Entrées réelles

transaction bancaire
signal marché
télémétrie GPS
capteur
payload API
français / anglais / JSON

pre

1. ENTRÉE DOMAINE

Banque / Trading / GPS / E-commerce annexe

2. DOMAIN STATE

BankState / TradingState / GpsDefenseAviationState / EcomState

3. AGENTS DOMAINE

observent chacun une facette du risque

4. AGENT VOTE

claim / confidence / severity_hint / unknowns / contradictions / risk_flags

5. DOMAIN AGGREGATE

scores métier + verdict candidat + evidence_refs

6. SIGMA / PROOF SIGNALS

stabilité dynamique / readiness / proof consistency

7. KERNEL X108

BLOCK > HOLD > ALLOW

8. ENVELOPE + AUDIT

decision_id / trace_id / reason_code / hash / replay

Règle

Agents = signalent
Agrégation = synthétise
Sigma = surveille
X108 = décide
OS3 = prouve

Message clé

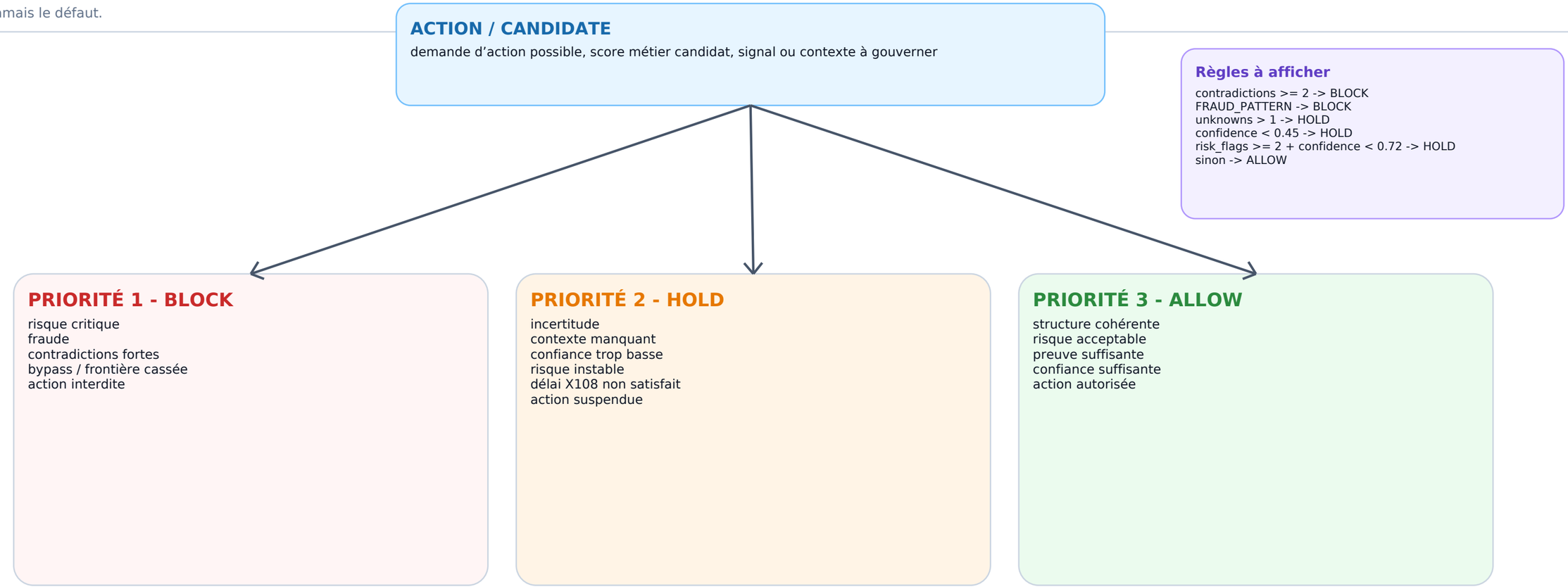
Le domaine propose. Les agents signalent. Sigma surveille. X108 décide. OS3 prouve.

À ne pas confondre

Ne pas faire croire que BankState, AgentVote, Aggregate ou Sigma peuvent autoriser.

Schéma 05 - BLOCK > HOLD > ALLOW

ALLOW n’est jamais le défaut.



Message clé

ALLOW arrive seulement quand BLOCK et HOLD n’ont rien trouvé.

Schéma 06 - HOLD / Temporal Lock

Déclencheurs HOLD

unknowns > 1
confidence < 0.45
risk_flags >= 2 + confidence < 0.72
elapsed_s < min_required_elapsed_s
stabilité insuffisante
preuve / attestation / trace insuffisante

1. ACTION CANDIDATE

demande d’action possible

2. X108 TEMPORAL / COHERENCE CHECK

elapsed_s / min_required_elapsed_s / confidence / unknowns / risk_flags / sigma_report

3. CONDITION NON SATISFAITE

temps insuffisant / contexte incomplet / trajectoire instable / confiance basse / preuve incomplète

4. HOLD

action non autorisée maintenant / aucune exécution réelle / attente / contexte / réévaluation

À ne pas faire

Ne pas présenter HOLD comme erreur, échec, indécision ou bug. Présenter HOLD comme verrou de sécurité, délai actif, refus temporaire d’action réelle.

Message clé

Une action possible n’est pas une action autorisée.

À ne pas confondre

HOLD != absence de décision.

Schéma 07 - Non-contournement

Couche	Droit d’autoriser ACT ?	Rôle
Agents domaine	NON	peuvent signaler
AgentVote	NON	signal structuré
DomainAggregate	NON	synthèse métier
Meta-agents	NON	contrôles transverses
Sigma	NON	surveillance stabilité
Projection française	NON	exposition readonly
Terminal / API	NON	surface
Mémoire / contexte	NON	contexte
Audit / replay	NON	preuve après coup
Kernel X108	OUI	ALLOW / HOLD / BLOCK

Message clé

Tout peut signaler. Seul X108 autorise. decision_authority = KX108_ONLY.

Schéma 08 - Entrée variable -> IR / alphabet interne

Entrées variables

français
anglais
JSON
transaction bancaire
signal marché
télémétrie GPS
capteur
payload API

1. ENTRÉE VARIABLE

français / anglais / JSON / transaction bancaire / signal marché / télémétrie GPS / capteur / payload API

2. NORMALISATION STRUCTURELLE

language / intent / request_type / domain_hint / constraints / risk_flags / boundary markers

3. IR / ALPHABET INTERNE

structure exploitable / pas phrase libre / pas intuition / pas génération libre

4. DOMAIN STATE

BankState / TradingState / GpsDefenseAviationState / EcomState

5. PIPELINE DOMAINE

agents -> votes -> aggregate -> X108

À ne pas dire

Le kernel parle français.
Le kernel est un LLM.
Le kernel comprend n’importe quoi directement.

Dire : l’entrée devient structure. La structure devient gouvernable.

Message clé

Le kernel ne comprend pas le français comme un chatbot. Il reçoit une structure gouvernable.

À ne pas confondre

IR / alphabet interne != génération libre.

PARTIE III - Domaines terrain

Banque, Trading, GPS et E-commerce : flux métier -> autorité X108.

Vue comparative des domaines

Banque	**Payload** transaction / fraude / identité	**BankState** état bancaire	**Agents** 12 agents	**Aggregate** AUTHORIZE / ANALYZE / BLOCK	**X108** ALLOW / HOLD / BLOCK
Trading	**Payload** marché / signal / exposition	**TradingState** état marché	**Agents** 17 agents	**Aggregate** BUY / SELL / HOLD	**X108** ALLOW / HOLD / BLOCK
GPS	**Payload** capteurs / trajectoire	**GpsState** état navigation	**Agents** 6 agents	**Aggregate** VALID / RECALC / DEGRADED / ABORT	**X108** ALLOW / HOLD / BLOCK

Message clé

Le domaine parle métier. X108 parle autorité.

Schéma 09 - Banque : flux complet

Payload bancaire

transaction_type
channel
amount
account_balance
counterparty_known
counterparty_age_days
behavior_shift_score
fraud_score
policy_limit
affordability_score
urgency_score
identity_mismatch_score
narrative_conflict_score
recovery_path_score
proof_ready
elapsed_s
min_required_elapsed_s

dan

1. PAYLOAD BANCAIRE

transaction_type / channel / amount / account_balance / counterparty_known / counterparty_age_days / behavior_shift_score / fraud_score / policy_limit / affordability_score / urgency_score / identity_mismatch_score / narrative_conflict_score / recovery_path_score / proof_ready / elapsed_s / min_required_elapsed_s

2. BankState

état métier bancaire exploitable

3. 12 AGENTS BANQUE

TransactionContextAgent / CounterpartyAgent / LiquidityExposureAgent / BehaviorShiftAgent / FraudPatternAgent / LimitPolicyAgent / AffordabilityAgent / TemporalUrgencyAgent / IdentityMismatchAgent / NarrativeConflictAgent / RecoveryPathAgent / BankProofAgent

4. AgentVote[]

claim / confidence / severity_hint / unknowns / contradictions / risk_flags / proposed_verdict / evidence_refs

5. aggregate_bank()

AUTHORIZE / ANALYZE / BLOCK + scores + proof_ready + deterministic

6. VERDICT MÉTIER CANDIDAT

AUTHORIZE / ANALYZE / BLOCK != autorisation finale

7. META-AGENTS + SIGMA

trace / ticket / attestation / replay / proof consistency / stability / readiness

8. KERNEL X108 -> ENVELOPE -> AUDIT

BLOCK > HOLD > ALLOW / CanonicalDecisionEnvelope / hash / replay

Métriques obligatoires

authorize_score
analyze_score
block_score
confidence
unknowns
contradictions
risk_flags
evidence_refs
proof_ready
deterministic

Message clé

La banque ne demande pas un oui/non direct. Elle produit un verdict métier candidat.

À ne pas confondre

AUTHORIZE != ALLOW. BLOCK métier != BLOCK kernel.

Schéma 09B - Banque : 12 agents

TransactionContextAgent

CounterpartyAgent

LiquidityExposureAgent

BehaviorShiftAgent

FraudPatternAgent

LimitPolicyAgent

AffordabilityAgent

TemporalUrgencyAgent

IdentityMismatchAgent

NarrativeConflictAgent

RecoveryPathAgent

BankProofAgent

Message clé

Les agents banque ne décident pas. Ils produisent des AgentVote.

Schéma 10 - Banque : métriques et cas X108

Cas banque	Signals / métriques	Verdict métier candidat	Sortie X108
Transaction cohérente	confidence haute	AUTHORIZE	ALLOW
Contrepartie inconnue	unknowns	ANALYZE	HOLD
Solde / liquidité fragile	affordability / liquidity	ANALYZE	HOLD
Fraude forte	FRAUD_PATTERN / fraud_score	BLOCK	BLOCK
Identité incohérente	identity_mismatch_score	ANALYZE / BLOCK	HOLD ou BLOCK selon signaux
Contradictions multiples	contradictions >= 2	BLOCK	BLOCK
Confiance trop basse	confidence < 0.45	ANALYZE	HOLD
Risques multiples	risk_flags >= 2 + conf < .72	ANALYZE	HOLD

Message clé

AUTHORIZE n’est pas ALLOW. ANALYZE n’est pas HOLD. Le domaine parle métier. X108 parle autorité.

Schéma 11 - Trading / Bourse : flux complet

Payload marché

symbol
prices
highs
lows
volumes
spreads_bps
event_risk_scores
sentiment_scores
predicted_returns
exposure
drawdown
order_book_imbalance
execution_cost_bps

1. PAYLOAD MARCHÉ

symbol / prices / highs / lows / volumes / spreads_bps / event_risk_scores / sentiment_scores / predicted_returns / exposure / drawdown / order_book_imbalance / execution_cost_bps

2. TradingState

état marché exploitable

3. 17 AGENTS TRADING

MarketDataAgent / LiquidityAgent / VolatilityAgent / MacroAgent / CorrelationAgent / EventAgent / MomentumAgent / MeanReversionAgent / BreakoutAgent / PatternAgent / SentimentAgent / PredictionAgent / PortfolioAgent / ExecutionQualityAgent / RegimeShiftAgent / PortfolioStressAgent / ProofConsistencyAgent

4. AgentVote[]

BUY / SELL / HOLD + confidence / severity / risk_flags / contradictions / unknowns / evidence_refs

5. aggregate_trading()

buy_score / sell_score / hold_score / confidence / unknowns / contradictions / risk_flags / evidence_refs

6. VERDICT MÉTIER CANDIDAT

EXECUTE_LONG / EXECUTE_SHORT / REVIEW / HOLD selon logique métier

7. META-AGENTS + SIGMA

stability / regime shift / proof signals

8. KERNEL X108 -> ENVELOPE -> AUDIT

vérifie risque, contradictions, confiance, instabilité

Message

Le trading peut proposer une direction. Mais proposer un trade n’est pas l’autoriser. Le kernel vérifie si l’action peut sortir.

Message clé

Le trading ne passe pas d’un signal marché à une action.

À ne pas confondre

BUY / SELL / HOLD métier != ALLOW / HOLD / BLOCK kernel.

Schéma 11B - Trading : 17 agents

MarketDataAgent

LiquidityAgent

VolatilityAgent

MacroAgent

CorrelationAgent

EventAgent

MomentumAgent

MeanReversionAgent

BreakoutAgent

PatternAgent

SentimentAgent

PredictionAgent

PortfolioAgent

ExecutionQualityAgent

RegimeShiftAgent

PortfolioStressAgent

ProofConsistencyAgent

Message clé

Aucun agent trading ne peut autoriser l’exécution.

Schéma 12 - Trading : métriques et cas X108

Cas trading	Signal / métrique	Verdict métier candidat	Effet X108
Signal cohérent	confidence suffisante + risque acceptable	BUY / SELL	ALLOW possible
Volatilité excessive	volatility spike / risk_flags	HOLD	HOLD
Régime instable	RegimeShiftAgent / instability	HOLD	HOLD / BLOCK selon gravité
Contradiction BUY / SELL	contradictions >= 2	HOLD / BLOCK	BLOCK
Exposition trop forte	exposure / portfolio stress	HOLD	HOLD
Drawdown	drawdown élevé	HOLD	HOLD
Event risk	event_risk_scores élevés	HOLD	HOLD
Coût exécution trop élevé	execution_cost_bps	HOLD	HOLD
Proof consistency insuffisant	ProofConsistencyAgent	HOLD	HOLD

Message clé

Un signal de marché peut proposer. Il ne peut pas autoriser l’exécution.

Schéma 13 - GPS / Défense / Aviation : flux complet

Payload GPS

gps_available
inertial_available
radio_available
trajectory_drift_score
source_conflict_score
time_skew_score
brownout_score
attestation_ready
rollback_possible

1. PAYLOAD GPS / DÉFENSE / AVIATION

gps_available / inertial_available / radio_available / trajectory_drift_score / source_conflict_score / time_skew_score / brownout_score / attestation_ready / rollback_possible

2. GpsDefenseAviationState

état navigation exploitable

3. AGENTS GPS

SourceAvailabilityAgent / TrajectoryIntegrityAgent / SourceConflictAgent / TimeSkewAgent / BrownoutAgent / AttestationReadinessAgent

4. AgentVote[]

TRAJECTORY_VALID / RECALC_TRAJECTORY / DEGRADED_NAVIGATION / ABORT_TRAJECTORY

5. Agrégation

trajectory_valid_score / recalc_score / degraded_score / abort_score / truth_score / sigma_score / mismatch_gap

6. Guard X108

si conflit source / dérive forte / brownout / time skew -> HOLD ou BLOCK

7. Envelope + preuve

trace / attestation / replay

Agents GPS

SourceAvailabilityAgent
TrajectoryIntegrityAgent
SourceConflictAgent
TimeSkewAgent
BrownoutAgent
AttestationReadinessAgent

Message clé

Le système compare GPS, inertiel, radio, trajectoire, temps, énergie, attestation, rollback.

À ne pas confondre

GPS available != action autorisée.

Schéma 14 - GPS : métriques et cas X108

Cas GPS	Signal	Verdict candidat	Effet X108
Sources cohérentes	gps / inertial / radio alignés	TRAJECTORY_VALID	ALLOW possible
Source conflict élevé	source_conflict_score	DEGRADED / ABORT	HOLD / BLOCK
Trajectory drift fort	trajectory_drift_score	RECALC / DEGRADED	HOLD
Time skew élevé	time_skew_score	RECALC / DEGRADED	HOLD
Brownout élevé	brownout_score	DEGRADED / ABORT	HOLD / BLOCK
Attestation non prête	attestation_ready=false	RECALC / DEGRADED	HOLD
Rollback impossible	rollback_possible=false	ABORT ou DEGRADED	BLOCK ou HOLD selon criticité

Message clé

Si les sources se contredisent ou si la trajectoire n’est pas stable, le système ne laisse pas passer l’action critique.

Schéma 15 - E-commerce : annexe domaine

Statut

Domaine annexe. À montrer comme extension possible, pas comme cœur prioritaire.

1. Ecom payload

commande / paiement / identité / panier / risque / contexte

2. EcomState

état métier e-commerce

3. Agents e-commerce

observent les facettes utiles

4. AgentVote

claim / confidence / risk_flags / evidence_refs

5. aggregate_ecom

verdict métier candidat

6. Guard X108

autorité finale

7. Envelope

trace / reason_code / audit

Règle

Même modèle : domaine -> agents -> aggregate -> X108 -> envelope.

Message clé

E-commerce prouve l’extensibilité, pas le cœur prioritaire.

PARTIE IV - Pipeline commun

Les briques communes rendent les domaines comparables.

Schéma 16 - AgentVote : unité atomique commune

Comment un agent transforme une observation en signal structuré ?

1. Observation métier

transaction / marché / trajectoire / capteur / contexte

2. Agent spécialisé

lit une facette du risque

3. AgentVote

claim / confidence / severity_hint / unknowns / contradictions / risk_flags / proposed_verdict / evidence_refs

4. DomainAggregate

les votes deviennent comparables et consolidables

Champs AgentVote

claim
confidence
severity_hint
unknowns
contradictions
risk_flags
proposed_verdict
evidence_refs

Message clé

Chaque agent transforme une observation métier en signal structuré comparable.

À ne pas confondre

AgentVote != décision finale.

Schéma 17 - DomainAggregate

Comment les votes deviennent un verdict métier candidat ?

1. AgentVote[]
votes du domaine

2. aggregate_bank / trading / gps / ecom
fonction d’agrégation par domaine

3. DomainAggregate
scores métier / confidence / unknowns / contradictions / risk_flags / evidence_refs / extra_metrics / proof_ready / deterministic

4. Verdict métier candidat
AUTHORIZE / ANALYZE / BLOCK ou BUY / SELL / HOLD ou VALID / RECALC / DEGRADED / ABORT

5. Vers X108
seul le kernel peut autoriser

À retenir

Le verdict métier candidat n’est pas une décision kernel. Chaque domaine parle métier. X108 parle autorité.

Message clé

Agrégation = synthèse, pas souveraineté.

Schéma 18 - Meta-agents

Qu’est-ce qui sécurise le passage entre métier et autorité ?

1. DomainAggregate

verdict candidat + scores métier

2. Meta-agents

UnknownsAgent / ConflictResolutionAgent / PolicyScopeAgent / TicketReadinessAgent / TraceIntegrityAgent / AttestationReadinessAgent / HumanOverrideEligibilityAgent / SeverityClassifierAgent / ReplayConsistencyAgent / ProofConsistencyAgent

3. Aggregate enrichi

unknowns / contradictions / risk_flags / policy_scope / ticket_required / trace_integrity / attestation_readiness / replay_consistency / proof_consistency / severity / human_override_eligibility

4. Vers Sigma + X108

contrôles transverses avant dernier mur

Rôle

Contrôler ce que les agents domaine ne couvrent pas directement : preuve, trace, ticket, policy, attestation, replay, cohérence, override humain, sévérité.

Message clé

Les meta-agents sécurisent mais ne deviennent pas autorité.

Schéma 19 - Sigma / proof signals

Où intervient la stabilité dynamique ?

1. Aggregate enrichi / décision candidate

market_verdict / confidence / severity / risk_flags / contradictions / unknowns / evidence_refs

2. Sigma Monitor

evaluate_step() / stability check / trajectory check / risk coherence / contradiction coherence / severity coherence

3. Sigma Output

sigma_report / sigma_override / stability_status / sigma_step / confidence_integrity / confidence_governance / confidence_readiness

4. Si instabilité

market_verdict = HOLD_STABILITY_ALERT / severity = S4 / sigma_override = true

5. Vers X108 / Envelope

qualification de stabilité, pas autorité finale

Message

Sigma est une couche de qualification de stabilité, pas une autorité finale. Sigma surveille. X108 décide.

Message clé

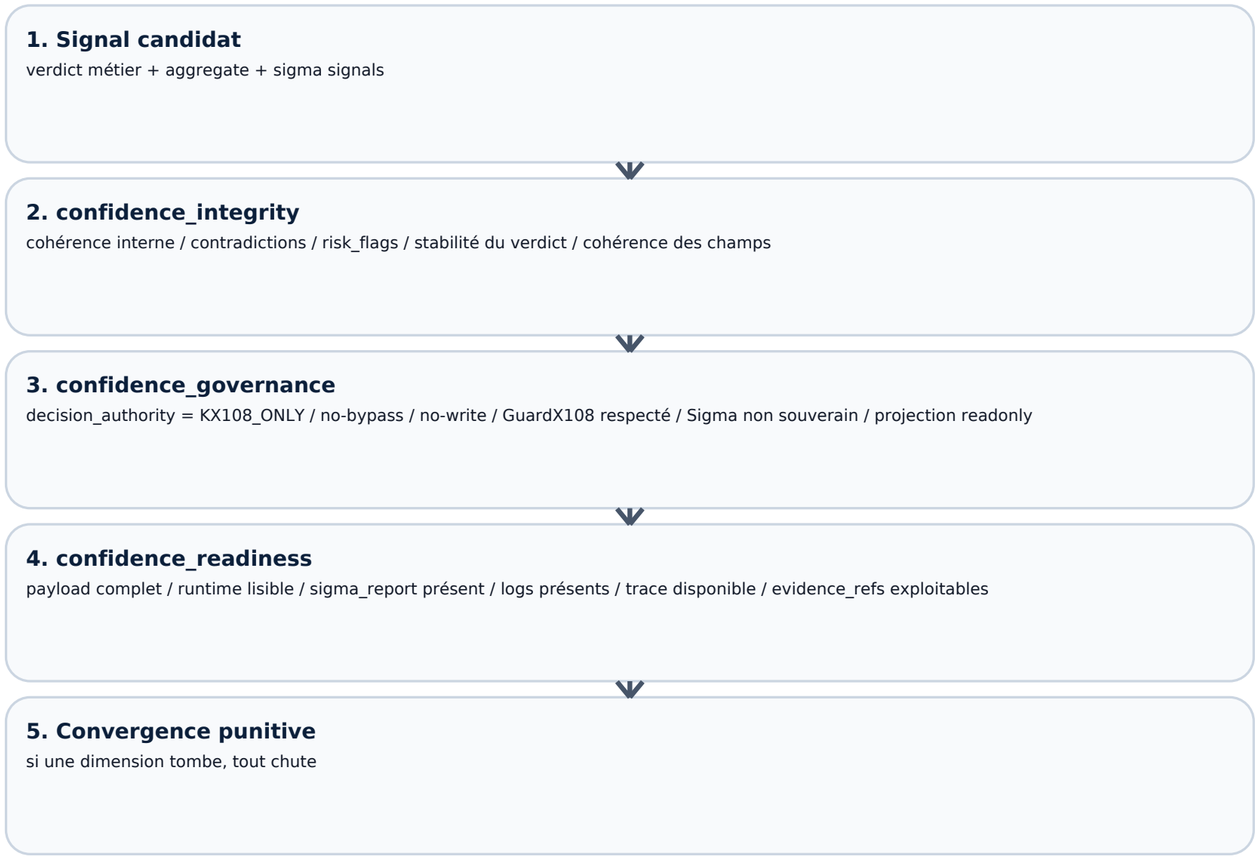
Sigma surveille la trajectoire décisionnelle.

À ne pas confondre

Sigma != X108.

Schéma 20 - Triple confidence

Pourquoi un bon score local ne suffit pas ?



Message clé

La confiance unique ne suffit pas. Un bon score local ne force jamais ALLOW.

Schéma 21 - Canonical Decision Envelope

Sous quelle forme sort une décision X108 ?



À retenir

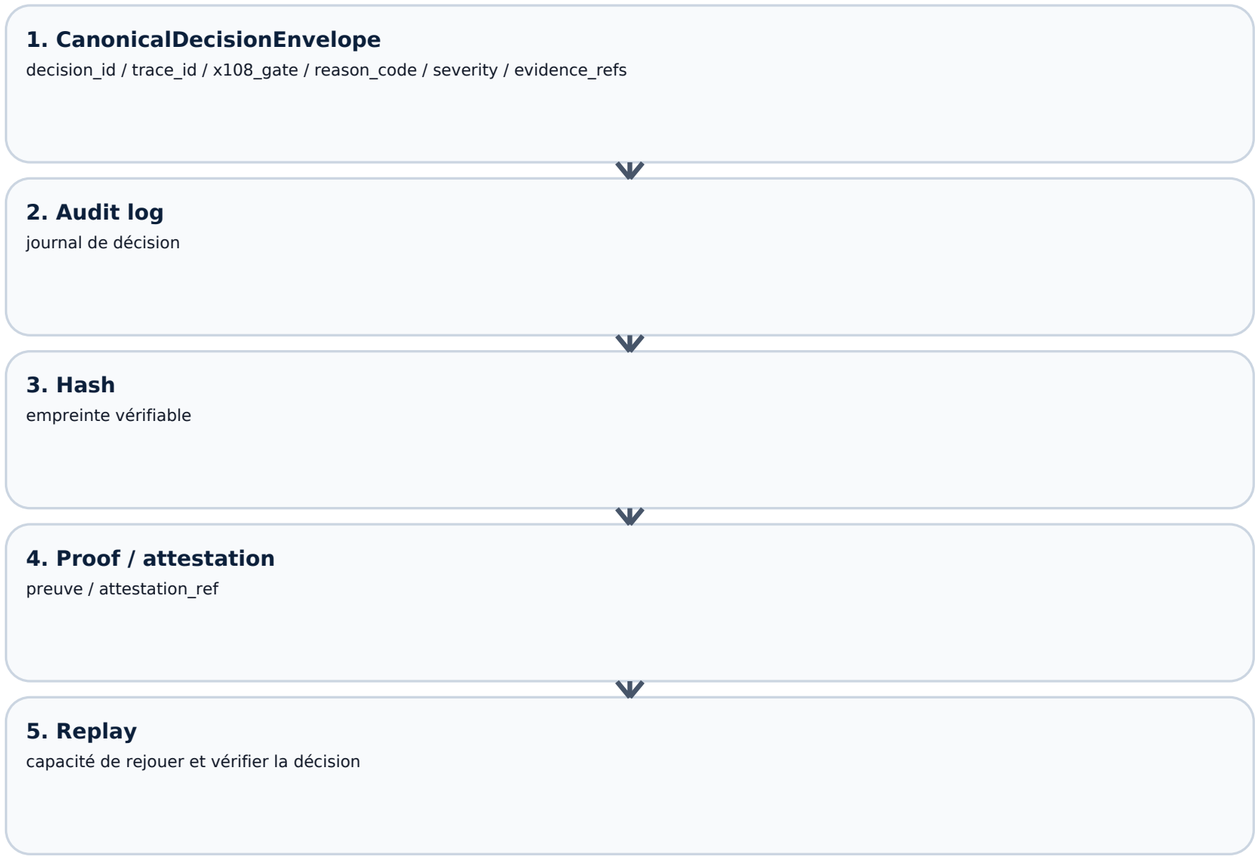
Le kernel ne sort pas une phrase, une opinion, une réponse libre ou une explication. Il sort un objet de décision canonique.

Message clé

La sortie X108 est un objet de décision, pas une prose.

Schéma 22 - Audit / hash / replay

Comment rendre la décision vérifiable ?



Message clé

Audit / hash / replay ne décident pas. Ils prouvent et vérifient.

Schéma 23 - Exposition humaine readonly

Comment exposer la décision sans redécider ?



Message clé

La projection française peut exposer. Elle ne peut pas autoriser.

À ne pas confondre

Exposition != décision.

Schéma 24 - OS / Runtime / Bus

Quelles couches portent la machination ?

1. OS4 - Interface / exposition readonly

payload / API / terminal / connecteur terrain / JSON / logs opérateur / projection FR si route prévue

2. OS1 - Structure / parse / validation

parse / validate / DomainState / action candidate / contract checks

3. OS2 - Métriques / scores / seuils

confidence / scores métier / thresholds / risk_flags / unknowns / contradictions / ACT-HOLD signals

4. Sigma / proof signals

stability / integrity / governance / readiness / sigma_report / sigma_override

5. OS0 - Kernel X108 / Guard

règles / invariants / seuils / temporal lock / BLOCK > HOLD > ALLOW

6. OS3 - Audit / preuve / replay

CanonicalDecisionEnvelope / trace / ticket / hash / replay

Message clé

OS4 expose. OS1 structure. OS2 mesure. Sigma surveille. OS0 décide. OS3 prouve.

PARTIE V - Socle formel / Synthèse kernel

Garder lisible le lien entre preuve, runtime et sortie gouvernée.

Schéma 25 - Lean 57 théorèmes

Basic
11 - seuils / ordres / propriétés de base / fondations logiques

TemporalKernel
5 - attente X108 / temporalité / non-anticipation / pas d'ACT avant condition

TemporalBridge
2 - pont temporalité <-> décision / cohérence entre temps et gate

Refinement
4 - raffinement décisionnel / passage structure -> décision

Consensus
10 - agrégation / priorité collective

ConsensusScratch
2 - scratch consensus / exploration formelle

Merkle
2 - evidence_ref / hash / audit_chain

CryptoAssumptions
10 - attestation / signature / anti-replay

Seal
1 - scellement de décision

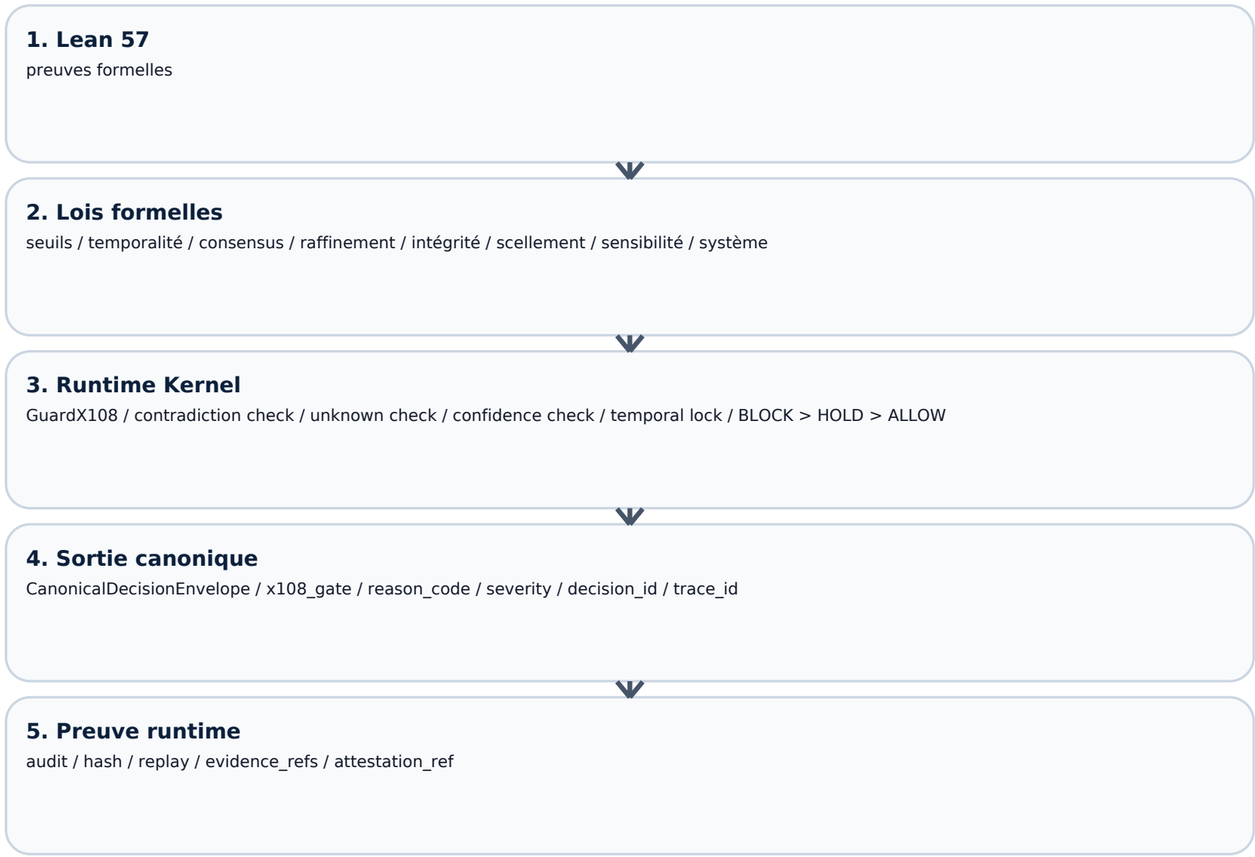
Sensitivity
3 - robustesse des seuils

SystemModel
7 - transition / auditabilité / cohérence globale

Message clé
Le kernel n'est pas seulement un pipeline logiciel. Il est adossé à un socle formel.

Schéma 26 - Lean -> Runtime Kernel

Comment les preuves Lean se relient au runtime X108 ?



Message clé

La preuve formelle ne remplace pas le runtime. Elle encadre les lois que le runtime applique.

À ne pas confondre

Lean != runtime. Runtime != preuve formelle.

Schéma 27 - Synthèse finale kernel

La preuve visuelle complète du kernel.

1. Entrée variable

texte / JSON / transaction / marché / capteur / télémétrie

2. IR / alphabet interne

structure gouvernable / risk_flags / constraints / domain markers / boundary markers

3. DomainState

BankState / TradingState / GpsDefenseAviationState / EcomState annexe

4. Agents domaine

Banque 12 / Trading 17 / GPS 6 / Ecom annexe

5. AgentVote

confidence / severity_hint / unknowns / contradictions / risk_flags / proposed_verdict / evidence_refs

6. DomainAggregate

market_verdict candidat / scores / proof_ready / deterministic

7. Meta-agents + Sigma

policy / trace / ticket / attestation / replay / stability

8. Kernel X108 -> Envelope -> Audit

BLOCK > HOLD > ALLOW / CanonicalDecisionEnvelope / audit / hash / replay

Message clé

La synthèse doit montrer la machination, pas des blocs décoratifs.

PARTIE VI - SRL / Session Registry Layer

Continuité sans surcharge. Mémoire sans bruit. Reprise sans relecture totale.

SRL-01 - Vue globale

Une session ne doit pas mourir dans l’historique.

1. SESSION RAW

conversation brute / idées / frictions / bugs / pépites / décisions / noms / liens / commandes / erreurs / validations

2. SESSION CARD

fiche structurée de reprise

3. TRIAGE AGENT

classe chaque fragment

4. ACTIVE / SEMI-ACTIVE / COLD / GHOST

statuts mémoire

5. SESSION INDEX

date / projet / type / module / concept / point d’arrêt

6. CONTEXT RELOAD PACK

reprise contextuelle sans relire tout

Statut SRL

Périphérique
Non moteur
Non décisionnel
Non connecté à la mémoire obsidienne pour l’instant
Branchable plus tard

Message clé

SRL = couche périphérique de continuité sessionnelle.

À ne pas confondre

SRL != mémoire centrale Obsidia. SRL != X108.

SRL-02 - Mémoire froide / fantôme

Ne pas tout mettre dans le contexte actif, mais ne rien perd

1. SESSION
session utilisateur / dev / audit / recherche / business / pitch / debug / documentation / freeze

2. RAW LOG
capture brute

3. TRIAGE AGENT
classe et réduit la charge contextuelle

4. ACTIVE MEMORY
chargée souvent

5. COLD MEMORY
chargée si pertinent

6. GHOST TABLE
jamais chargée par défaut

7. RETRIEVAL POLICY
demande explicite / échec recherche active / collision sémantique / audit

8. CONTEXT PACK FINAL
contexte utile, sans surcharge

Loi

SUPPRESSION INTERDITE
MISE EN FROID AUTORISÉE
RÉACTIVATION TRAÇABLE

NO_DELETE -> COLD_SHIFT -> SYMBOLIC_TRACE -> CONDITIONAL_RECALL

Message clé
Ne jamais supprimer une pépite potentielle. Réduire sa présence, pas son existence.

SRL-03 - Session card / fiche de reprise

Chaque session devient une fiche structurée, pas un bloc perdu dans l’historique.

1. Session brute

conversation longue ou travail opérateur

2. Session fiche

point de départ / point d’arrêt / décisions / pépites / frictions / fichiers / modules / concepts / éléments à reprendre

3. Triage mémoire

actif / semi-actif / froid / fantôme

4. Index

jour / projet / module / couche / concept

5. Reprise contextuelle

retrouver la bonne fiche / charger seulement le bon bloc / reprendre au bon point

Message clé

À la reprise, le système ne relit pas tout l’historique. Il recharge la fiche pertinente.

SRL-04 - Positionnement SRL

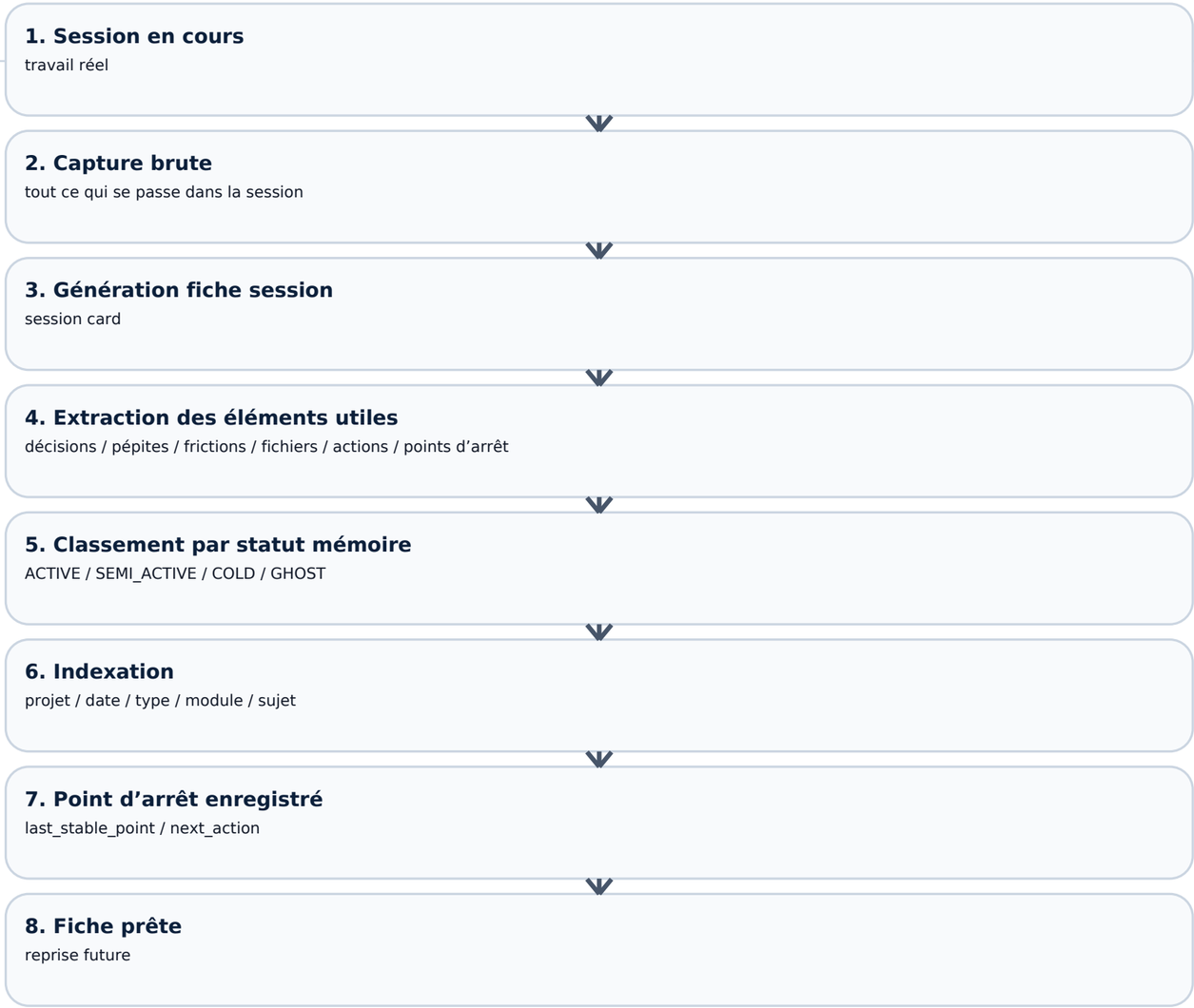
Obsidia Core	**Core** lois / noyau / X108	**Plus tard seulement** interface contrôlée
SRL	**Session Registry Layer** couche périphérique	**Rôle** range / indexe / retrouve / fournit contexte
Utilisateurs	**Sessions** utilisateur / dev / audit / recherche / pitch	**Workbench** reprise / export / rappel

Message clé

SRL ne décide pas. SRL ne modifie pas le noyau. SRL ne touche pas aux lois moteur.

SRL-05 - Flux complet SRL

Comment une session devient-elle reprenable ?



Message clé

Le but est de reprendre sans relire toute la conversation brute.

SRL-06 - Les 4 zones mémoire

ACTIF

décision validée
point d’arrêt
module en cours
bloc à continuer
action prochaine

SEMI-ACTIF

idée secondaire
hypothèse
piste non prioritaire
fragment lié à un projet

FROID

détail utile plus tard
ancienne piste
contexte secondaire
discussion non urgente

FANTÔME

bruit possible
doublon
fragment faible
idée incertaine
élément non prioritaire

Message clé

Ne pas tout charger. Ne pas supprimer. Tout fragment écarté doit garder une trace.

SRL-07 - Registre latéral des éléments mis de côté

Même ce qui est écarté reste traçable.

1. Fragment faible / doublon / bruit possible

candidat à cold ou ghost

2. Registre séparé

ghost_registry.jsonl / cold_index.csv / session_discard_table.json

3. Champs

fragment_id / session_id / date / status / reason / symbolic_hash / keywords / reactivation_policy

4. Réactivation conditionnelle

demande explicite / absence de résultat / audit / collision sémantique

5. Trace conservée

pas de suppression réelle

Exemple source

fragment_id
session_id
date
status: GHOST
reason: doublon / bruit / faible priorité / hors sujet temporaire
symbolic_hash
keywords
reactivation_policy

Message clé

Même ce qui est mis de côté reste auditable.

SRL-08 - Pseudo-hash symbolique

Retrouver sans avoir les mots exacts.

1. Fragment froid / fantôme

élément non prioritaire mais conservé

2. Signature de sens

hash symbolique != hash cryptographique pur

3. Exemple

SESSION:2026-06-04 / TYPE:MEMORY_LAYER / DOMAIN:USER_SESSION / INTENT:CONTEXT_CONTINUITY / STATUS:COLD / GLYPH:NO_DELETE_SIDE_MEMORY

4. Utilité

retrouver / reconnecter / réactiver / auditer

Rôle

retrouver sans les mots exacts
reconnecter une idée ancienne
réactiver une pépite oubliée
auditer ce qui a été mis de côté

Message clé

HASH SYMBOLIQUE = signature de sens.

SRL-09 - Format canonique d’une fiche session

Identité

session_id
date
session_type
project
main_topic

Entrée

entry_context
previous_session_ref
starting_point
user_intent

Sortie

exit_point
last_stable_point
next_action
unresolved_threads

Blocs

active_blocks
semi_active_blocks
cold_fragments
ghost_fragments

Travail

decisions
validated_items
open_threads
frictions
pepites

Références

files_links
commands_or_actions
reactivation_rules
status

Status values

OPEN
FROZEN
TO_SORT
TO_BRANCH
ARCHIVED

Message clé

Une fiche SRL doit donner un point de reprise, pas seulement un résumé.

SRL-10 - Indexation multi-entrées

Date
jour / période

Projet
Obsidia / X108 / Brody / Graphiti / SRL

Type de session
utilisateur / dev / audit / recherche / business / pitch / debug / documentation / freeze / stratégie / mémoire / test

Module
kernel / SRL / Graphiti / Bank / Trading / GPS

Concept
mémoire froide / ghost / X108 / HOLD / boundaries

Statut
OPEN / FROZEN / TO_SORT / TO_BRANCH / ARCHIVED

Point d’arrêt
last_stable_point / next_action

Pépite
signal fort à réactiver

Décision
décision validée / à trancher

Fichier
path_or_url / rôle

Message clé
Une session ne doit pas être relue. Elle doit être retrouvée.

SRL-11 - Reprise de session

Comment redémarrer sans tout relire ?

1. Identifier le sujet actuel

projet / module / concept / intention

2. Chercher les fiches liées

index par date / projet / type / module / concept / point d’arrêt

3. Charger seulement les fiches pertinentes

active_blocks + contexte nécessaire

4. Reprendre depuis exit_point

last_stable_point

5. Récupérer active_blocks

bloc à continuer / action prochaine

6. Ignorer cold / ghost sauf besoin

pas de surcharge par défaut

Règle stricte

READ_ALL_HISTORY = interdit par défaut
LOAD_SESSION_CARD = autorisé
DEEP_RECALL = conditionnel

Message clé

Le système recharge la fiche pertinente au lieu de relire toute la conversation brute.

SRL-12 - Réactivation conditionnelle

Condition 1
l’utilisateur le demande

Condition 2
la mémoire active ne suffit pas

Condition 3
un audit exige de vérifier

Condition 4
une collision sémantique apparaît

Condition 5
un projet lié revient

Condition 6
une pépite ancienne redevient utile

Message clé
Le rappel profond est conditionnel, jamais automatique par défaut.

SRL-13 - Session Triage Agent

Le petit agent de tri prépare la continuité.

1. Session Triage Agent

périphérique

2. Actions autorisées

classer / ranger / marquer / indexer / mettre en froid / mettre en fantôme / préparer la fiche de reprise

3. Interdictions

ne décide pas / ne supprime pas / ne modifie pas le noyau

4. Sortie

fiche session + index + context reload pack

Boundaries

Il ne décide pas.
Il ne supprime pas.
Il ne modifie pas le noyau.
Il prépare la continuité.

Message clé

Le Session Triage Agent range la mémoire ; il ne gouverne pas l’action.

SRL-14 - Limites V0

Où ne pas brancher SRL en V0 ?

1. Pas de branchement

mémoire obsidienne centrale / Graphiti profond / X108 décisionnel / kernel / runtime moteur / lois internes complètes

2. V0 reste sur

fiche session / index / classement / reprise / mémoire froide / mémoire fantôme

3. Rôle V0

continuité sans surcharge / mémoire sans bruit / reprise sans relecture totale / classement sans perte

Message clé

SRL V0 reste périphérique autonome.

À ne pas confondre

SRL V0 != mémoire centrale ni X108.

SRL-15 - Branchage futur

Quel futur branchement, sous conditions ?

1. SRL
session cards / index / cold / ghost / recall

2. Mémoire Obsidienne
plus tard seulement

3. Graphiti / Atlas / Workbench
navigation / recherche / export

4. OS3 audit
traçabilité / auditabilité

5. OS4 interface
interface de rappel / workbench

Conditions

SRL ne décide pas
SRL ne modifie pas le noyau
SRL fournit du contexte
SRL reste traçable
SRL peut être auditée

Message clé
Le branchage futur doit préserver la non-souveraineté SRL.

SRL-16 - Architecture V0 figée

Le flux figé de SRL V0.



Message clé

SRL V0 = continuité sessionnelle, tri, rappel, audit léger.

SRL-17 - Audit SRL dans obsidia-x108-proofs

Rectification : audit cible = obsidia-x108-proofs uniquemen

1. periphery/brody_memory_readonly/
zone cible existante

2. session_presave_buffer_readonly
base session / presave

3. project_intake_capture_buffer_readonly
capture projet

4. brody_session_memory_adapter / runtime
adaptateurs session memory

5. FOUNDATION_B_SESSION_MEMORY
base existante

6. Audit SRL V0
formalisation + métriques + gaps readonly

Statut réel

BASE SRL PRÉSENTE
COUCHE SRL NON CANONISÉE
FRONTIÈRES READONLY À RÉALIGNER
MÉTRIQUES SRL À AJOUTER

Message clé

SRL n’est pas à créer de zéro. Elle existe en germe dans Brody Memory Readonly.

SRL-18 - Métriques SRL

Ledger metrics

SRL_LEDGER_FOUND
SRL_LEDGER_RECORD_COUNT
SRL_JSONL_TRACE_VALID
SRL_SESSION_ID_PRESENT
SRL_PREVIOUS_EVENT_HASH_PRESENT
SRL_HASH_CHAIN_VALID
SRL_LAST_TURN_CONTEXT_FOUND

Follow-up metrics

SRL_FOLLOWUP_SUPPORTED
SRL_FOLLOWUP_REQUESTED
SRL_FOLLOWUP_RESOLVED
SRL_TOPIC_EXTRACTED
SRL_LAST_CONTEXT_EXTRACTED
SRL_USED_BY_API_NOW

Presave buffer

SRL_PRESAVE_POINTER_COUNT
SRL_POINTER_HASH_VALID
SRL_REFERENCED_PATH_COUNT
SRL_EVENT_HASH_CHAIN_VALID
SRL_MANUAL_VALIDATION_REQUIRED

Triage metrics

SRL_ACTIVE_COUNT
SRL_SEMI_ACTIVE_COUNT
SRL_COLD_COUNT
SRL_GHOST_COUNT
SRL_ACTIVE_RATIO
SRL_COLD_RATIO
SRL_GHOST_RATIO
SRL_TRIAGE_CONFIDENCE
SRL_CONTEXT_OVERLOAD_SCORE

Reprise metrics

SRL_EXIT_POINT_FOUND
SRL_EXIT_POINT_QUALITY
SRL_RELOAD_PACK_CREATED
SRL_RELOAD_PACK_FRAGMENT_COUNT
SRL_RELOAD_PACK_MINIMALITY
SRL_DEEP_RECALL_USED
SRL_GHOST_REACTIVATION_COUNT

Non-souveraineté

readonly=true
memory_write=false
graphiti_write=false
neo4j_write=false
memory_decision=false
allowed_to_decide=false
emits_act=false
emits_verdict=false
kernel_mutation=false
x108_runtime_binding=false
x108_merge=false
decision_authority=KX108_ONLY

Message clé

Les métriques SRL doivent prouver la continuité, la réduction du bruit, le rappel contrôlé et la non-souveraineté.

PARTIE VII - Synthèse fusionnée

X108 autorise. SRL rappelle. Les deux restent séparés.

X108 vs SRL : deux chemins séparés

ux ré

Chemin décisionnel X108

Domaine
payload

Agents
signalent

Aggregate
synthétise

Sigma
surveille

X108
autorise

Envelope
prouve

Chemin mémoire SRL

Session
brute

Card
fiche

Triage
classe

Cold/Ghost
conserve

Recall
recharge

Pack
contexte

Message clé

X108 gouverne l’action. SRL gouverne la continuité contextuelle.

Tableau final des rôles

Composant	Rôle	Peut décider ?	Peut ACT ?	Statut
Kernel X108	autorité	oui	selon verdict	souverain
Sigma	surveillance	non	non	advisory / proof signal
Agents	observation	non	non	signal
DomainAggregate	synthèse métier	non	non	verdict candidat
Audit	preuve	non	non	trace / replay
SRL	mémoire sessionnelle	non	non	périphérique
Cold / Ghost	conservation hors contexte actif	non	non	rappel conditionnel

Message clé

Tout peut signaler ou conserver. Seul X108 autorise.

Carte finale fusionnée

Document complet en une seule machination lisible.

1. Entrée réelle

Domaine critique OU session utilisateur

2. Si action critique

payload -> DomainState -> Agents -> Aggregate -> Sigma -> X108 -> Envelope -> Audit

3. Si continuité mémoire

session raw -> session card -> triage -> active/semi/cold/ghost -> index -> reload pack

4. Point commun

trace / index / auditabilité / preuve de chemin

5. Différence

X108 produit ALLOW / HOLD / BLOCK. SRL produit Context Reload Pack.

6. Conclusion

autoriser != rappeler ; décision != mémoire ; preuve != publication brute

Message clé

Le PDF doit aider à comprendre sans exposer le moteur propriétaire complet.

ANNEXES COMPACTES

Détails conservés sans casser les flux principaux.

Annexe A - Boundaries X108 / SRL

X108

decision_authority = KX108_ONLY
BLOCK > HOLD > ALLOW
no-bypass
temporal lock
reason_code
CanonicalDecisionEnvelope

Périphérie

readonly = true
emits_act = false
emits_verdict = false
memory_write = false
kernel_mutation = false

SRL

ne décide pas
ne supprime pas
ne modifie pas le noyau
ne charge pas ghost par défaut
deep recall conditionnel

P78

DO_NOT_PUBLISH
source pack local-only
public proof != moteur propriétaire
démo != runtime souverain

Message clé

Les boundaries doivent être visibles, pas cachées dans une note.